

SDEV2171

Lesson 16: State Management With Context

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- explain how Context helps multiple screens use one shared value
- create a Context and provider for one shared feature
- place the provider at the shared Expo Router layout level
- access shared state with a custom hook that uses `useContext`
- refactor a duplicated or parameter-passed value into one shared source of truth

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson Roadmap

1. Reconnect lesson 15’s mismatch to the Context solution
2. Define provider, consumer, and shared source of truth
3. Place the provider in the route layout
4. Move one feature into Context
5. Confirm both screens stay synchronized
6. Checkpoint and lesson-17 handoff

WE ARE ESSENTIAL TO ALBERTA | NAIT

Starting Point From Lesson 15

- keep the same local backend and `{ id, label }` study-step data
- keep the same three-route Expo Router flow
- keep the same selected-step feature

What changes today

- the selected-step list will stop living inside one screen
- the copied route-parameter plan will be removed
- both screens will use one shared provider value

WE ARE ESSENTIAL TO ALBERTA | NAIT

Context In Plain Language

- Context is a React tool for sharing one value across multiple components or screens
- it helps multiple places read the same source of truth
- it reduces the need for copied mutable data or long prop chains

Important rule

- Context is for shared state
- local-only UI values can still stay local

WE ARE ESSENTIAL TO ALBERTA | NAIT

Four Pieces Today

1. context
2. provider
3. consumer
4. shared hook

Quick definitions

- `context` : the shared value channel
- `provider` : the component that supplies the shared value
- `consumer` : any component that reads or updates the shared value
- `shared hook` : a helper that makes Context usage easier and safer

WE ARE ESSENTIAL TO ALBERTA | NAIT

Where The Provider Goes

- not inside only one screen
- not inside the plan screen alone
- at the shared layout level

Why

- both the details screen and the plan screen must sit inside the same provider

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson-15 Problem Versus Lesson-16 Fix

Before

- details screen owned local selected steps
- plan screen received a copied snapshot
- the screens drifted apart

After

- provider owns selected steps
- both screens read and update the same value
- the screens stay synchronized

WE ARE ESSENTIAL TO ALBERTA | NAIT

Build Order Today

1. create the Context file
2. wrap the route tree with the provider
3. move the selected-step state into the provider
4. replace local screen state with the shared hook
5. retest both screens

WE ARE ESSENTIAL TO ALBERTA | NAIT

Keep These Values Local

- loading state for one fetch
- error message for one fetch
- temporary draft input text
- one open or closed card

Shared in this lesson

- the selected review-plan steps

WE ARE ESSENTIAL TO ALBERTA | NAIT

Visible Clue To Confirm Success

1. add two study steps on the details screen
2. open the shared plan screen
3. remove one item there
4. go back to the details screen

Expected clue

- the details screen count now matches the updated shared plan

WE ARE ESSENTIAL TO ALBERTA | NAIT

Common Wrong Turns

- placing the provider too low so one screen sits outside it
- moving every local value into Context
- forgetting to replace the old copied parameter path
- using the hook outside the provider

WE ARE ESSENTIAL TO ALBERTA | NAIT

Midpoint Check

By the middle of class, students should know:

- where the provider file lives
- where the provider wraps the route tree
- which value moved into shared state
- which value still stays local

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

Students should be ready to show

- one provider wrapped around the shared route tree
- one shared selected-step value used in more than one screen
- one update from the plan screen reflected back on the details screen
- one local-only value that did not move into Context

Status language

- `resolved` : the shared feature works across screens and the provider path is clear
- `partial` : part of the provider or hook path works, but the full shared flow is incomplete
- `blocked` : the provider or hook setup is still broken or unclear

WE ARE ESSENTIAL TO ALBERTA | NAIT

What We Learned Today

- Context provides one shared source of truth for a cross-screen feature
- provider placement matters in the Expo Router scaffold
- multiple screens can stay synchronized by reading the same shared value
- local state still has a job for screen-specific UI

WE ARE ESSENTIAL TO ALBERTA | NAIT

Next Step

- keep the Context example small and clear
- keep local-only UI state out of shared state unless it truly belongs there
- prepare to combine shared state with forms and submit flows in lesson 17

Ready after lesson 16 means

- `resolved` : the cross-screen selected-step feature is shared correctly
- `partial` : one provider or consumer issue still needs repair
- `blocked` : the shared-state path still needs instructor-guided recovery

WE ARE ESSENTIAL TO ALBERTA | NAIT